

中山大学计算机学院

人工智能

本科生实验报告

课程名称: Artificial Intelligence

学号	22336203	姓名	沈苇杭
----	----------	----	-----

一、实验题目

Lab 3-3: 博弈树搜索: 编写一个中国象棋 AI, 要求使用 alpha-beta 剪枝算法, 实现两个 AI 对弈。其中, 一方由内置 AI 控制, 一方由人类或自己编写的 AI 控制。算法要求实现红黑双方的转换。本实验为代码补全, 只需要实现 *MyAI* 类的 *get_next_step* 方法和 *main* 函数中的 AI 落子、红黑双方转换。

二、实验内容

算法原理

- 博弈树搜索:** 在博弈中, 双方玩家都有各自的利益, 并且都会按照自己的利益来改变当前的局面。博弈中可能出现的情况和玩家能采取的行动离散且有限。在零和博弈中, 一方赢了多少, 另一方就会输掉对应的量。
- minimax 搜索:**
 - 前提条件: 假设对方总是能做出最优的举动。最优举动可以最大化自身利益, 最小化对手利益。倘若在某些情况下对方不能做出最优举动, 那么或许有比 minimax 搜索更好的算法。
 - 算法原理: 游戏的两个玩家分别为 max 方和 min 方。Max 方试图让效益值最大, min 方试图让效益值最小。游戏的每个终止状态都有一个效益值。根据终止状态的效益值和 max/min 方, 可以计算出每个中间状态的效益值, 进而得出游戏进行的路线。
 - 算法步骤:
 - 构建完整的博弈树: 根节点表示起始状态, 边表示可能的行动, 叶子节点表示终止状态。每个终止状态都有一个效益值。
 - 反向传播效益值: 根据子节点效益值和当前层是 max 层还是 min 层, 计算出当前节点的效益值。重复计算, 直到算出所有节点的效益值。
 - 找出游戏路线: 根据每个节点的效益值和 min/max, 得出行动路线。
- alpha-beta 剪枝:** minimax 算法会计算整棵博弈树, 但这是没有必要的。
 - alpha: max 结点的 alpha 值是它被遍历过的子节点的最高效用值, min

- 结点的 alpha 值是目前为止它的兄弟节点的最高效用值。
2. Beta: max 结点的 beta 值是目前为止它的兄弟节点的最低效用值, min 结点的 beta 值是它被遍历过的子节点的最低效用值。
 3. 剪枝: 若 $\alpha \geq \beta$, 剪枝。

关键代码展示

1. get_next_step:

```
def get_next_step(self, chessboard: ChessBoard, depth):
    best_move = list()
    b_score = float('-inf')
    alpha = float('-inf')
    beta = float('inf')
    minimax = 0

    for chess in chessboard.get_chess():
        if chess.team == self.team:
            active_move = chessboard.get_put_down_position(chess)
            for new_row, new_col in active_move:
                old_row = chess.row
                old_col = chess.col
                new_pos = chessboard.chessboard_map[new_row][new_col]
                chessboard.chessboard_map[new_row][new_col] =
chessboard.chessboard_map[old_row][old_col]
                chessboard.chessboard_map[old_row][old_col] = None

                chessboard.chessboard_map[new_row][new_col].update_position(new_row, new_col)
                score = self.minimax(chessboard, depth, alpha, beta, minimax)
                chessboard.chessboard_map[old_row][old_col] =
chessboard.chessboard_map[new_row][new_col]

                chessboard.chessboard_map[old_row][old_col].update_position(old_row, old_col)
                chessboard.chessboard_map[new_row][new_col] = new_pos
                if score > b_score:
                    b_score = score
                    best_move.append(old_row)
                    best_move.append(old_col)
                    best_move.append(new_row)
                    best_move.append(new_col)

    return best_move

def minimax(self, chessboard: ChessBoard, depth, alpha, beta, minimax):
    if depth == 0:
        return self.evaluate_class.evaluate(chessboard)
```

```

if minimax:
    max_score = float('-inf')
    for chess in chessboard.get_chess():
        if chess.team == self.team:
            max_active_move = chessboard.get_put_down_position(chess)
            for new_row, new_col in max_active_move:
                old_row, old_col = chess.row, chess.col
                new_pos = chessboard.chessboard_map[new_row][new_col]
                chessboard.chessboard_map[new_row][new_col] =
chessboard.chessboard_map[old_row][old_col]
                chessboard.chessboard_map[old_row][old_col] = None

chessboard.chessboard_map[new_row][new_col].update_position(new_row, new_col)
                score = self.minimax(chessboard, depth - 1, alpha, beta, 0)
                chessboard.chessboard_map[old_row][old_col] =
chessboard.chessboard_map[new_row][new_col]

chessboard.chessboard_map[old_row][old_col].update_position(old_row, old_col)
                chessboard.chessboard_map[new_row][new_col] = new_pos
                max_score = max(max_score, score)
                alpha = max(alpha, score)
                if beta <= alpha:
                    break
            return max_score

else:
    min_score = float('inf')
    for chess in chessboard.get_chess():
        if chess.team != self.team:
            min_active_move = chessboard.get_put_down_position(chess)
            for new_row, new_col in min_active_move:
                old_row = chess.row
                old_col = chess.col
                old_chess_in_new_pos =
chessboard.chessboard_map[new_row][new_col]
                chessboard.chessboard_map[new_row][new_col] =
chessboard.chessboard_map[old_row][old_col]
                chessboard.chessboard_map[old_row][old_col] = None

chessboard.chessboard_map[new_row][new_col].update_position(new_row, new_col)
                score = self.minimax(chessboard, depth - 1, alpha, beta, 1)
                chessboard.chessboard_map[old_row][old_col] =
chessboard.chessboard_map[new_row][new_col]

```

```

chessboard.chessboard_map[old_row][old_col].update_position(old_row, old_col)
        chessboard.chessboard_map[new_row][new_col] =
old_chess_in_new_pos
        min_score = min(min_score, score)
        beta = min(beta, score)
        if beta <= alpha:
            break
    return min_score

```

在方法的开始，设置 best_score, alpha, beta 的初始值。遍历棋盘上己方的棋子，对于每个符合规则的行动，通过递归到达博弈树的叶子节点，然后求解非叶子节点并剪枝，最终得到游戏路径。值得注意的是，在 max 层，遍历己方棋子（最初也是 max 层）；而在 min 层，遍历对方棋子。

2. AI 落子：

```

elif not game.show_win and not game.show_draw and game.AI_mode and
game.get_player() == chessAi.team:
    step = step + 1
    if game.back_button.is_repeated():
        print("你是", game.user_team, "方", game.get_player(), "方获胜")
        game.set_win(game.get_player())
    else:
        """
        if step < 60:
            depth = 3
        else:
            depth = 4
        """

    # AI 预测下一步
    cur_row, cur_col, nxt_row, nxt_col = chessAi.get_next_step(chessboard,3)
    # 选择棋子
    ClickBox(screen, cur_row, cur_col)
    # 下棋子
    chessboard.move_chess(nxt_row, nxt_col)
    # 清理「点击对象」
    ClickBox.clean()
    # 检测落子后，是否产生了"将军"功能
    if chessboard.judge_attack_general(game.get_player()):
        print("将军...")
    # 检测对方是否可以挽救棋局，如果能挽救，就显示"将军"，否则显示"胜利"
    if chessboard.judge_win(game.get_player()):
        print("你是", game.user_team, "方", game.get_player(), "方获胜")
        game.set_win(game.get_player())
    else:

```

```

        # 如果攻击到对方，则标记显示"将军"效果
        game.set_attack(True)
    else:
        if chessboard.judge_win(game.get_player()):
            print("你是", game.user_team, "方", game.get_player(), "方获胜")
            game.set_win(game.get_player())
            game.set_attack(False)

        if chessboard.judge_draw():
            print("和棋...")
            game.set_draw()

    # 落子之后，交换走棋方
    game.back_button.add_history(chessboard.get_chessboard_str_map())
    game.exchange()

```

AI 落子的代码和已给出的几乎一致。但需要注意的是，己方落子同样需要发生在未胜利、未和棋的条件下，否则可能出现棋局结束后仍然走子的情况。

3. 红黑双方转换：

```

4. def main():
    # 第一部分：创建棋盘，画面，计时器，游戏，AI
    # 初始化 pygame
    player = input("如果您想成为红方（先手），请输入 1; 如果您想成为黑方（后手），请输入 0")
    pygame.init()
    # 创建用来显示画面的对象（理解为相框）
    screen = pygame.display.set_mode((750, 667))
    # 游戏背景图片
    background_img = pygame.image.load("images/bg.jpg")
    # 游戏棋盘
    # chessboard_img = pygame.image.load("images/bg.png")
    # 创建棋盘对象
    chessboard = ChessBoard(screen)
    # 创建计时器
    clock = pygame.time.Clock()
    # 创建游戏对象（像当前走棋方、游戏是否结束等都封装到这个对象中）
    game = Game(screen, chessboard)
    if player == '0':
        game.user_team = 'b'
        game.computer_team = 'r'
    else:
        game.user_team = 'r'
        game.computer_team = 'b'

```

```
game.back_button.add_history(chessboard.get_chessboard_str_map())
# 创建 AI 对象
ai = ChessAI(game.computer_team)
chessAi = ChessAI(game.user_team)
```

采用了一个简单的 if...else 语句来实现红黑双方选择。根据提示输入对应的数字，即可作为双方参与对弈。

创新点

1. 修改了 print 语句的输出：
原先 print 语句输出为：获胜.....
修改后，print 语句会输出玩家所在的阵营和当前获胜阵营，有助于玩家判断自己是哪一方，以及自己是否获胜。
2. 尝试对搜索过程进行优化：尝试通过记录已走的步数，在残局时增加搜索深度，但这样会导致搜索时间过长，因此没有进行这一优化。

三、实验结果及分析

1. 实验结果展示

1. 作为红方：

```
E:\学习\Python3.7\python.exe "E:\学习\校内课程学习资料\大二\人工智能\homework 3-3 中国象棋\象棋code\AIchess_win\main.py"
pygame 2.5.2 (SDL 2.28.3, Python 3.7.9)
Hello from the pygame community. https://www.pygame.org/contribute.html
如果您想成为红方（先手），请输入1；如果您想成为黑方（后手），请输入0
```

```
旧位置： 8 4 新位置： 9 4 13步双方没有互吃棋子
旧位置： 8 2 新位置： 9 2 14步双方没有互吃棋子
将军....
旧位置： 9 4 新位置： 8 4 15步双方没有互吃棋子
旧位置： 9 2 新位置： 8 2 16步双方没有互吃棋子
将军....
旧位置： 8 4 新位置： 9 4 17步双方没有互吃棋子
你是 r 方， b 方获胜
```

作为红方时，游戏正常进行，红方失败。

2. 作为黑方：

```
E:\学习\Python3.7\python.exe "E:\学习\校内课程学习资料\大二\人工智能\homework 3-3 中国象棋\象棋code\AIchess_win\main.py"
pygame 2.5.2 (SDL 2.28.3, Python 3.7.9)
Hello from the pygame community. https://www.pygame.org/contribute.html
如果您想成为红方（先手），请输入1；如果您想成为黑方（后手），请输入0
```

```
旧位置: 0 4 新位置: 1 4 20步双方没有互吃棋子
旧位置: 1 6 新位置: 3 7 21步双方没有互吃棋子
将军....
旧位置: 1 4 新位置: 0 4 22步双方没有互吃棋子
旧位置: 3 7 新位置: 1 6 23步双方没有互吃棋子
将军....
你是 b 方, b 方获胜
```

作为黑方时，黑方获胜。

2.评测指标展示及分析

1. 红黑双方转换：可以通过从键盘输入“0”或“1”来指定这一局将成为红方还是黑方，具体可以在运行中查看输入提示。
2. 作为红方：可以完成一次棋局，在这次棋局中红方失败。
3. 作为黑方：可以完成一次棋局，在这次棋局中黑方获胜。
4. 算法的时间复杂度：假设棋盘上有 n 个棋子。对这 n 个棋子进行遍历，然后进行剪枝搜索。对每一个棋子进行剪枝搜索的时间复杂度为 $O(n^{\text{depth}/2})$ ，遍历棋子的复杂度为 $O(n)$ ，则 `get_next_step` 方法的时间复杂度为 $O(n^{(\text{depth}/2 + 1)})$ 。

四、参考资料

1. [最清晰易懂的 MinMax 算法和 Alpha-Beta 剪枝详解-CSDN 博客](#)
2. 理论课 ppt
3. [Alpha-beta 剪枝算法-网易伏羲 \(163.com\)](#)
4. [基于 alpha-beta 剪枝技术的五子棋 - 知乎 \(zhihu.com\)](#)